

ANALYSIS & VISUALIZATION OF TIME SERIES DATA IN PYTHON

DAY 1

Dr. –Ing. Mohamed-Ali Tnani

December 2025



My name is ...

I work as ...

I work with **time-series** in the context of ...

I work with **Python** in the context of ...

From this seminar, I expect ...

What interests me most is ...



What will we cover?

Day 1

- **Understanding Time-Series Data**

Patterns · Stationarity · Autocorrelation

- **Preprocessing Time-Series Data**

Resampling · Normalization · Windowing

- **Overview on Classical Methods**

AR/MA · ARIMA · Prophet



What will we cover?

Day 2

- **Overview on Deep Learning Architectures for Time-Series Data**

RNNs · LSTMs · GRUs · TCNs · Transformer/TFT

- **Training & Evaluation techniques for Time-Series Data**

Metrics · Avoiding leakage · Hyperparameter-Tuning



PAUSE

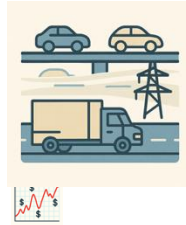
- 10:45 : Coffee Break
- 12:30 – 13:30: Lunch Break



TIME-SERIES DATA ARE EVERYWHERE

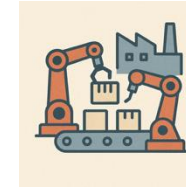
Finance

- Forecasting stock prices
- Credit Risk modeling



Logistics & Transport

- Traffic flow prediction
- Estimating delivery times



Manufacturing

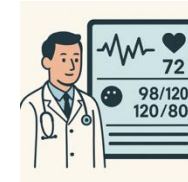
- Production line monitoring
- Predictive maintenance



Tech

- Network traffic analysis
- User activity monitoring

Time-Series Applications



Healthcare & Life Science

- Disease progression tracking
- Patient vital signs monitoring



Energie

- Electricity load forecasting
- Oil pipeline monitoring



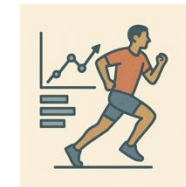
Retail & E-Commerce

- Sales forecasting
- Dynamic pricing



Environment

- Weather forecasting
- Air quality monitoring



Sports

- Athlete performance tracking
- Audience ratings over time

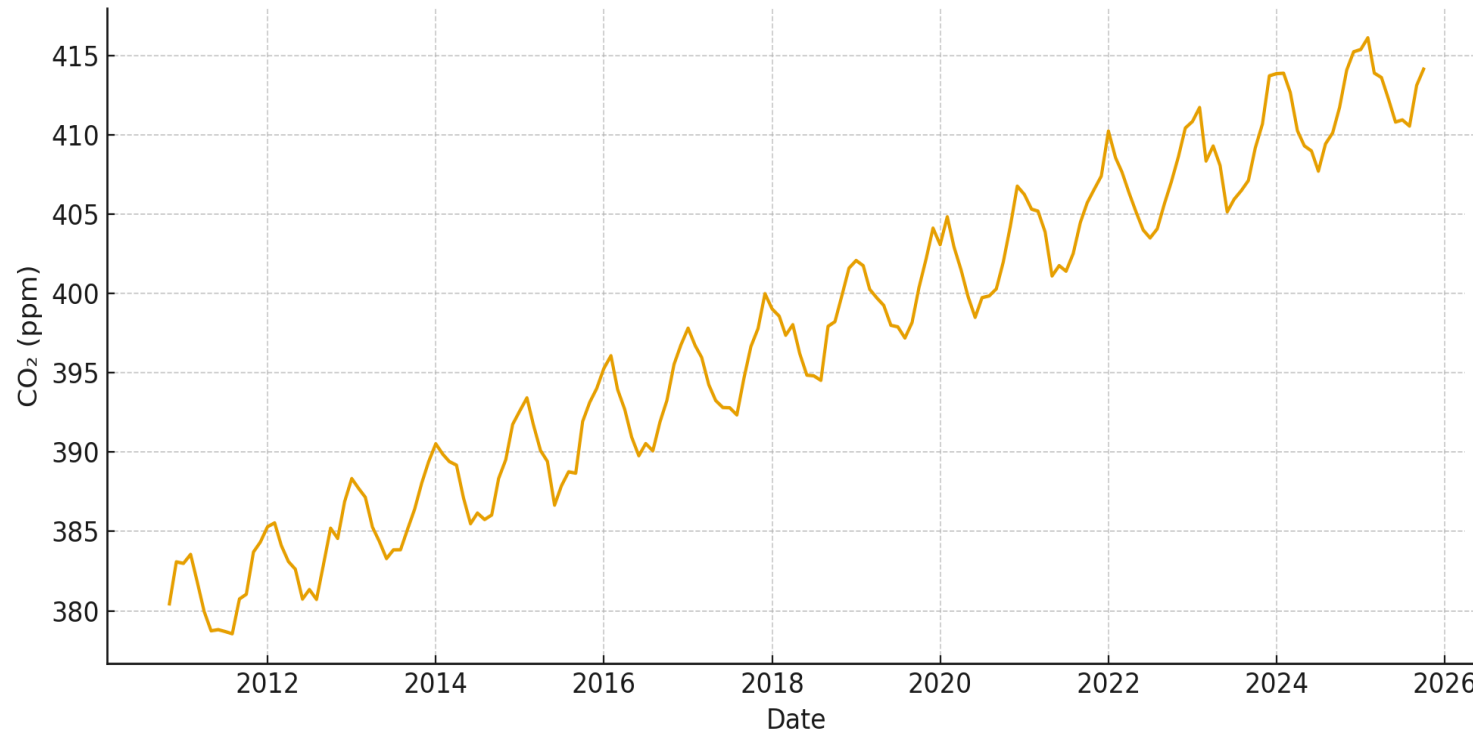
EXAMPLE 1: STOCK PRICES



What patterns/ characteristics do you see?



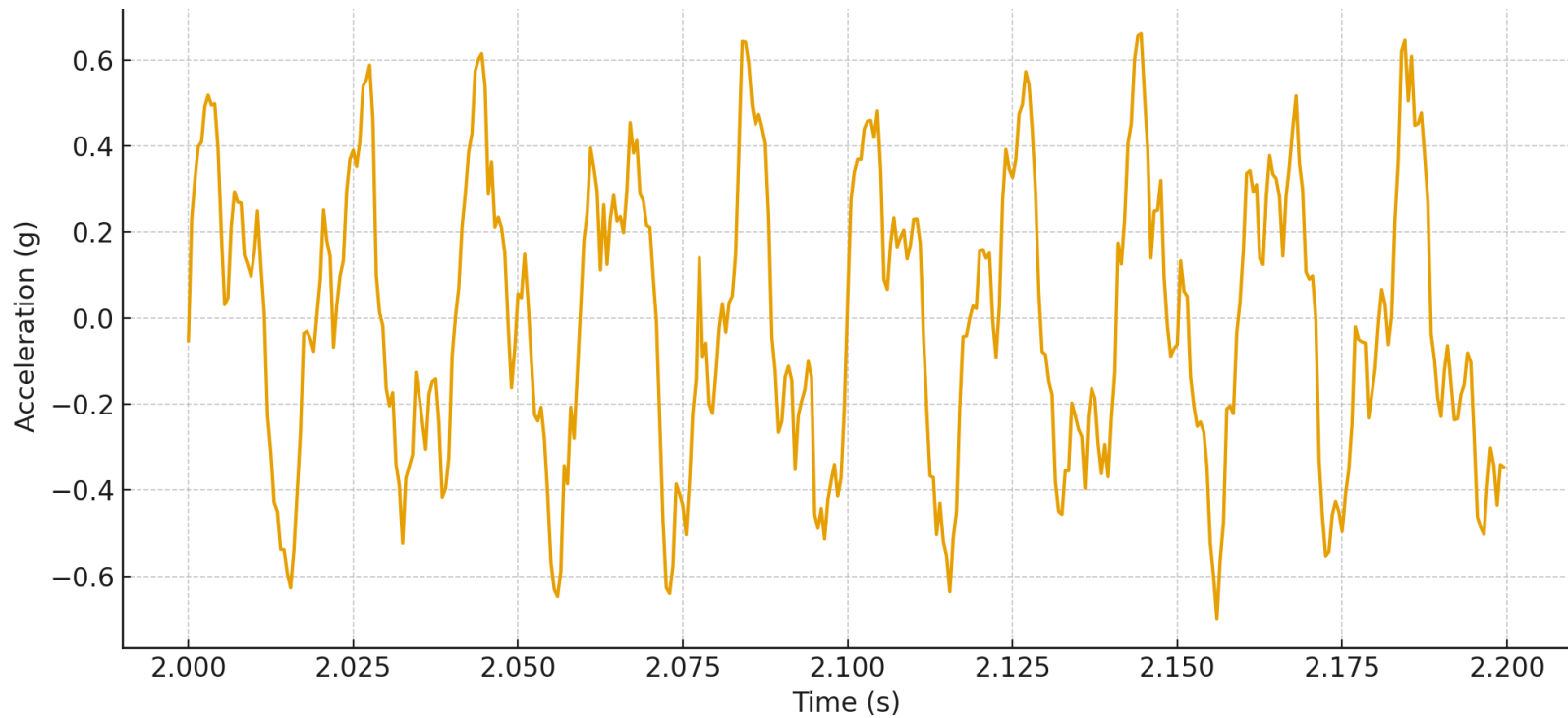
EXAMPLE 2: CO2 EMISSION



What patterns/ characteristics do you see?



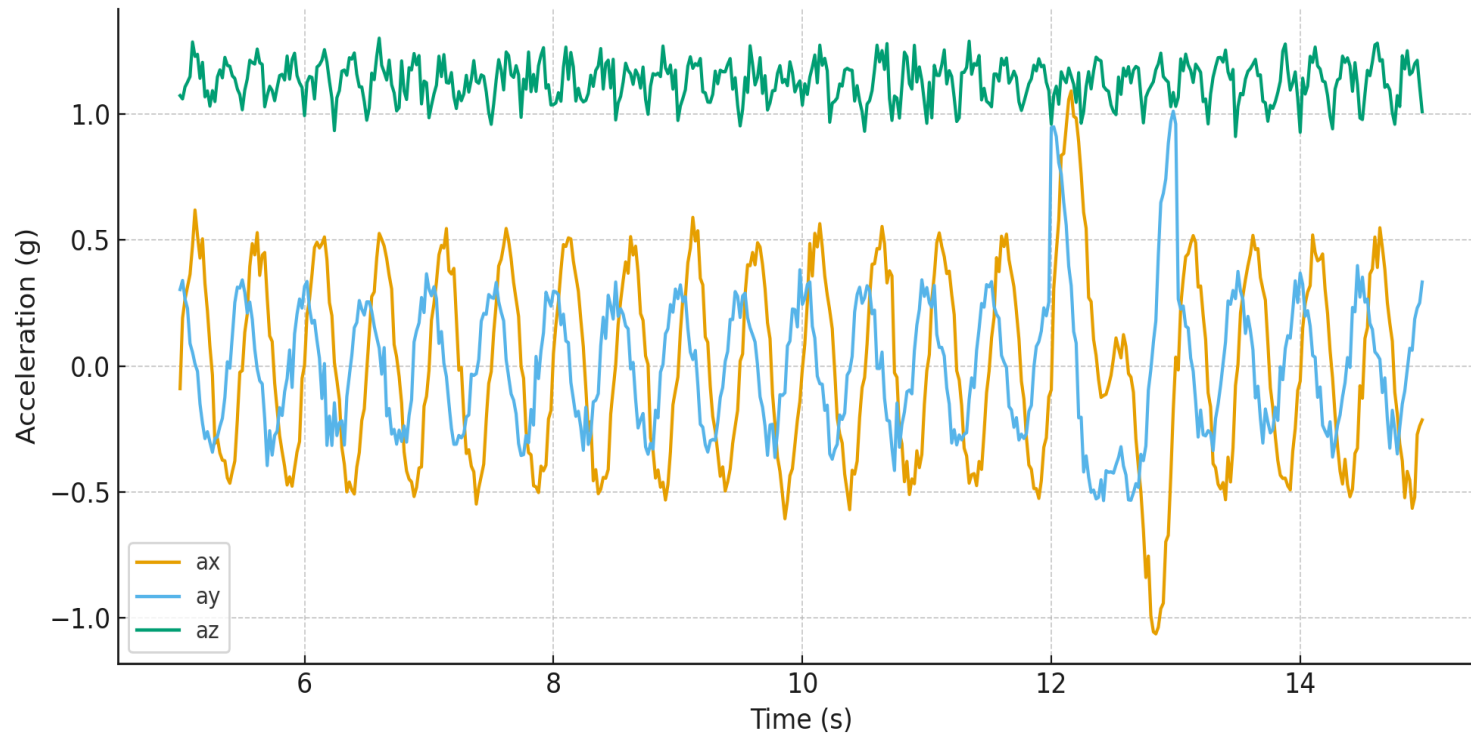
EXAMPLE 3: MACHINE HEALTH INDEX



What patterns/ characteristics do you see?



EXAMPLE 4: SMARTPHONE ACCELEROMETER

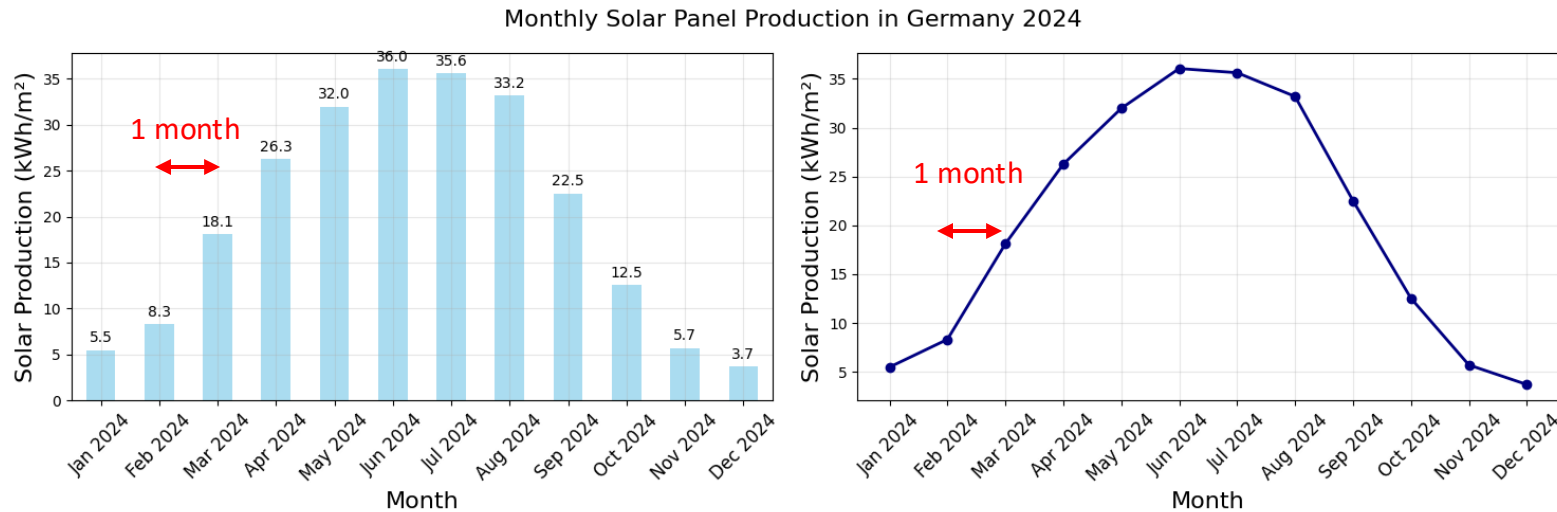


What patterns/ characteristics do you see?

- Milliseconds sampling rate
- Each sensor has its own dynamics
- Multiple sensor recording



WHAT ARE TIME-SERIES DATA?



A **Time-Series** is a sequence of data points recorded in time order, usually at equally spaced intervals

$\{x_t\}_{t=1}^T$ $t \in \{1, 2, \dots, T\}$, where

x_t = observed data point at time t ,

t = time index (usually equally spaced),

T = total number of observations.

“order matters -> each observation depends on previous ones.”



VISUALIZING TIME-SERIES DATA

Why plot time-series data?

- First step to understand structure and behavior.
- Reveals patterns such as trend, seasonality, spikes, and noise.
- Helps detect anomalies, missing values, and regime changes.
- Essential before choosing preprocessing steps or models.

What to look for:

- **Overall shape:** upward/downward trend?
- **Repeating cycles:** daily/weekly/yearly patterns?
- **Volatility changes:** does variance increase or decrease?
- **Breakpoints:** sudden shifts or structural changes?

matplotlib

 **seaborn**

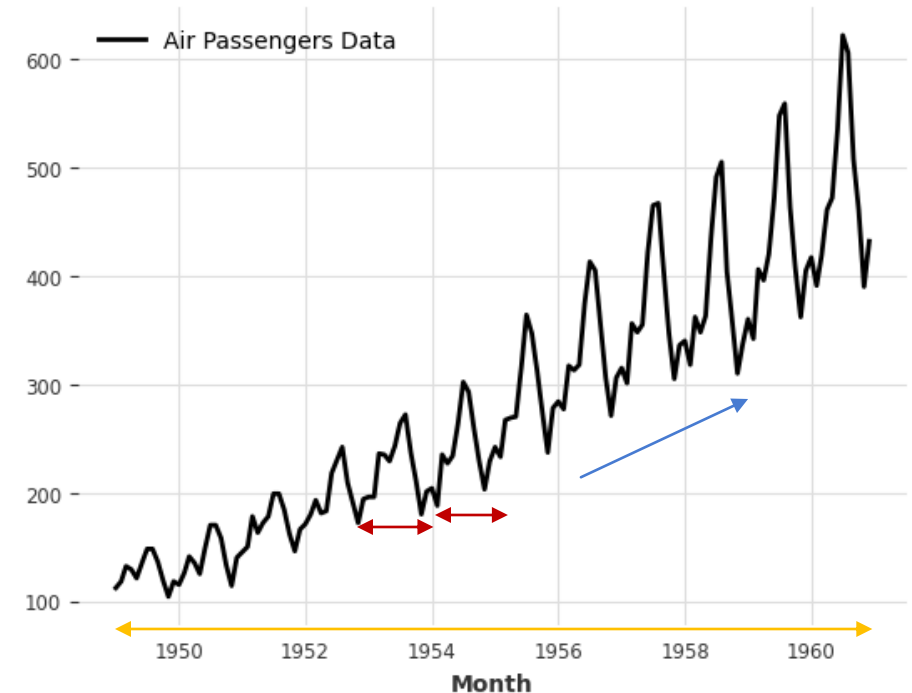
 **plotly**



CONCEPTS IN TIME-SERIES

Time series are often **composed** of distinct underlying patterns that help explain their behavior:

- **Trend** – long-term direction in the data (upward or downward movement)
- **Seasonal** – repeating patterns at fixed intervals (daily, weekly, yearly)
- **Cyclic** – irregular long-term fluctuations without a fixed period (climate cycles and energy markets)
- **Residual / Noise** – randomness not explained by other components (measurement noise, anomalies, and unmodeled variability)



SEASONAL DECOMPOSITION

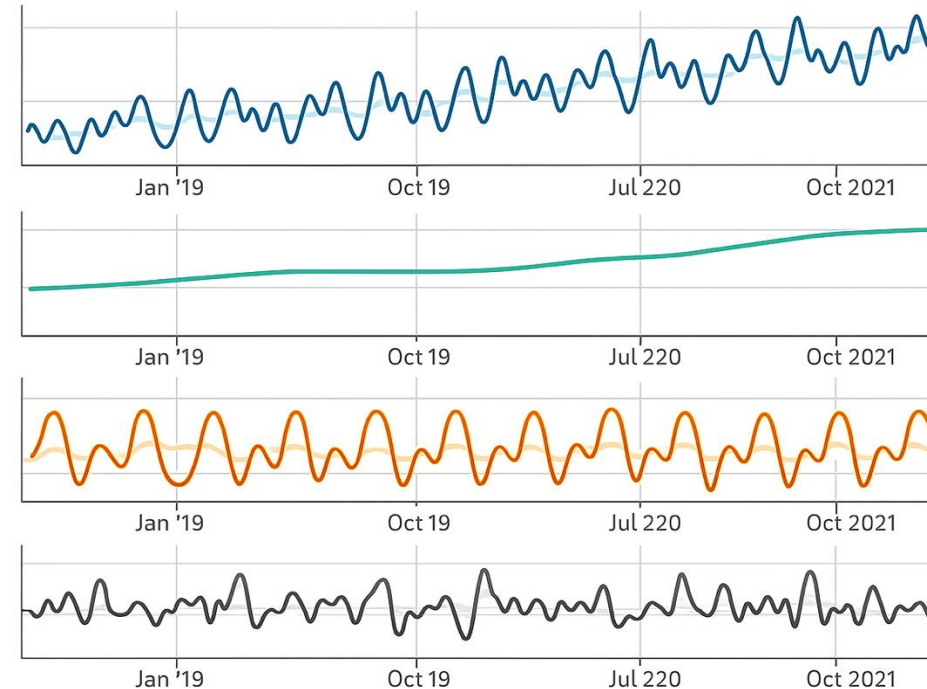
Seasonal decomposition is a technique used in time series analysis to break down a time series into its underlying components: trend, seasonality, and residual.

Two approaches:

- Additive decomposition: $x = T + S + R$
- Multiplicative decomposition: $x = T \times S \times R$

where T is the trend (cycle) component, S is the seasonal component, and R is the residual component.

Seasonal Decomposition



Original

Trend

Seasonal

Residual

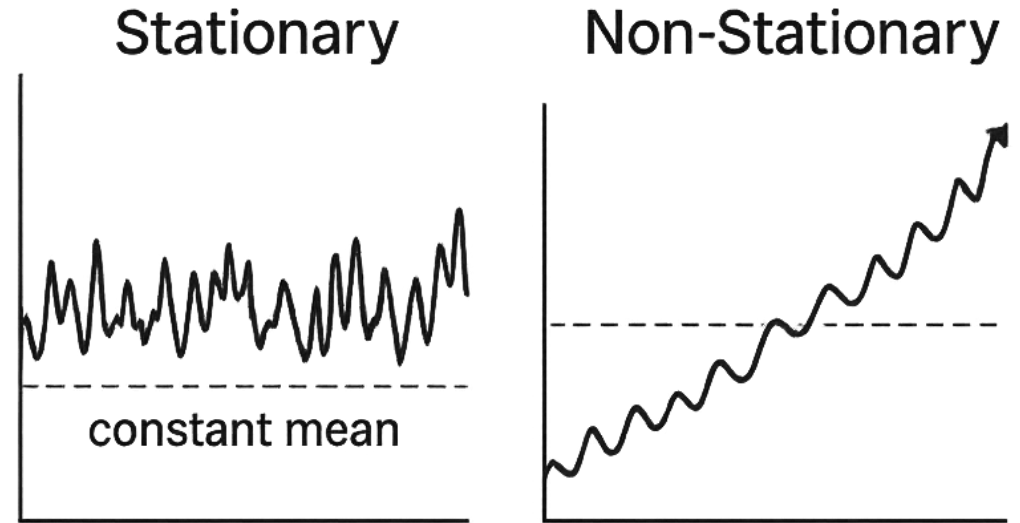


STATIONARITY IN TIME-SERIES DATA

A time series is **stationary** when its statistical properties do **not change over time**.

→ mean, variance, and autocorrelation remain constant.

- **Non-stationary** series show trends, seasonality, or changing variability.



Why?

- Many methods assume stable statistical properties
- Without it, patterns are harder to model, compare, and forecast



STATIONARITY: HOW TO DEAL WITH IT?

To convert trendy data to “stationary” data, we can use

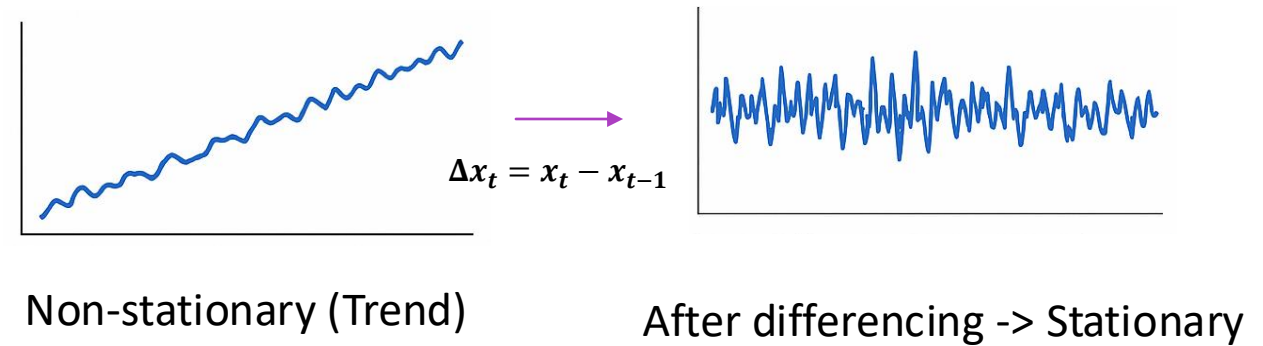
Differencing.

Many time series are non-stationary due to trends or seasonal patterns.

Differencing removes these trends by subtracting consecutive values:

$$\Delta x_t = x_t - x_{(t-1)}$$

Stabilizes the series → more constant means and variances

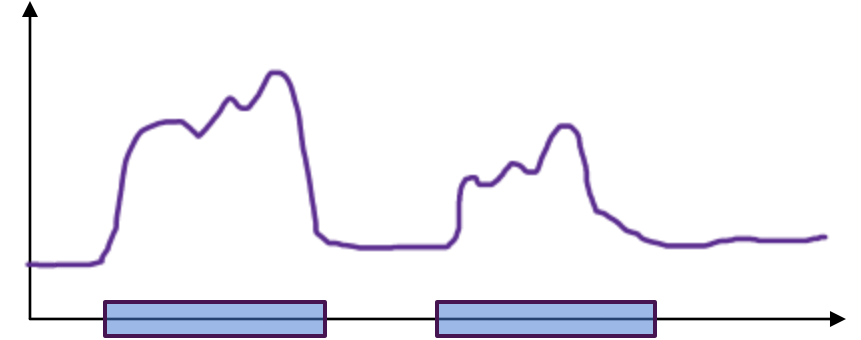


AUTOCORRELATION

Autocorrelation measures how strongly a time series is related to its own past values.

Interpretation:

- **Positive autocorrelation** → values follow similar direction
- **Negative autocorrelation** → values tend to reverse direction
- **Near zero** → weak or no relationship



🤔 **Why?**

- Reveals temporal structure in the data
- Helps detect seasonality and lag relationships

“Does the value at time t depend on earlier values $t-1, t-2, \dots$?”



AUTOCORRELATION FUNCTION (ACF)

Function to measure the strength of dependencies at different lag distances

Helps selecting **MA(q)** terms in ARIMA

Typical ACF patterns:

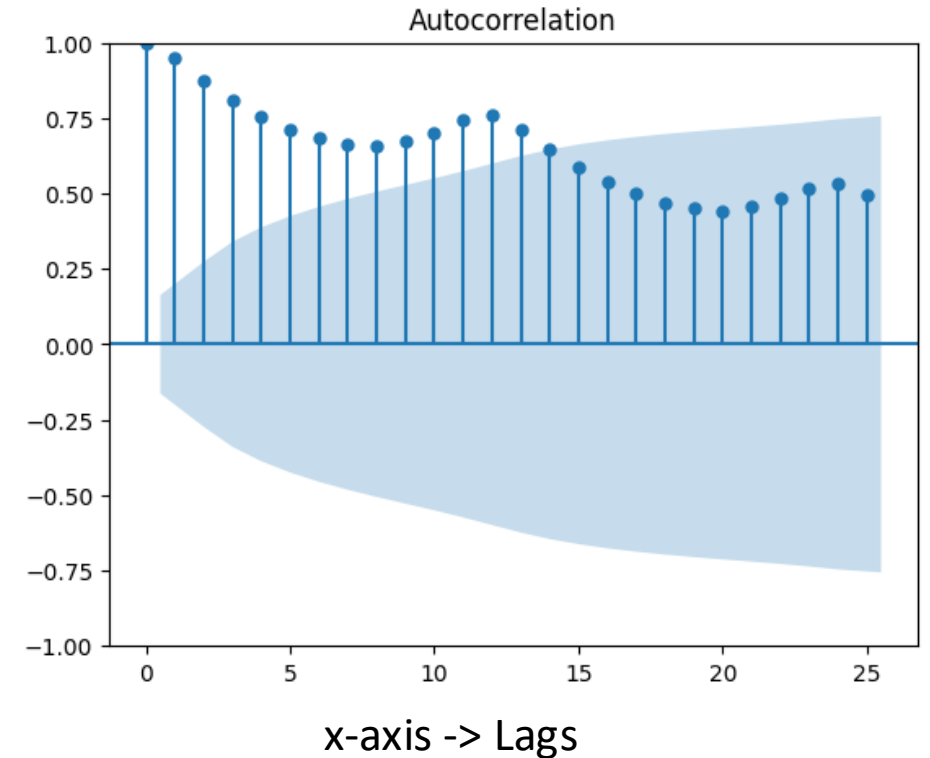
- Slow decay → non-stationary or trend
- Sharp cutoff → MA process
- Seasonal peaks → repeating patterns

$$\rho_k = \frac{\sum_{t=k+1}^T (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^T (x_t - \bar{x})^2}$$

Where:

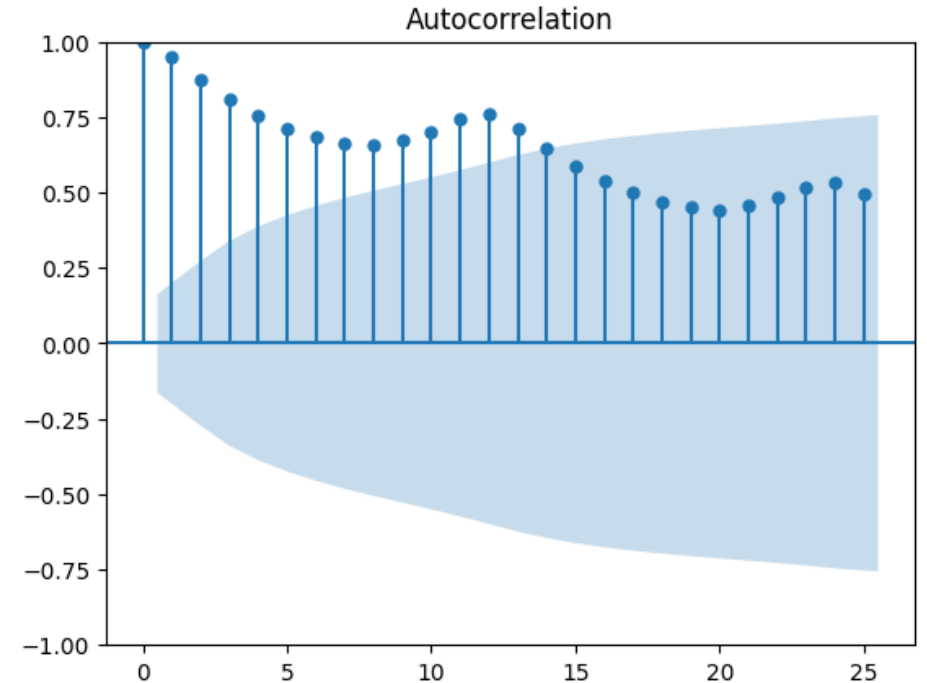
- ρ_k = autocorrelation at lag k
- x_t = value at time t
- $\bar{x}$ = mean of the series
- T = length of the series

y-axis -> Correlations coefficients



AUTOCORRELATION FUNCTION (ACF)

- The correlation coefficient at **lag 0 is always 1**, as it represents the correlation between an observation and itself.
- ACF values **close to 1** indicate a **strong positive** correlation, while values **close to -1** indicate a **strong negative** correlation.
- ACF values **close to 0** indicate that there is **no significant correlation** between the observations with this lag.



PARTIAL AUTOCORRELATION FUNCTION (PACF)

Function measures the correlation between the series and its lagged values **after removing intermediate correlations.**

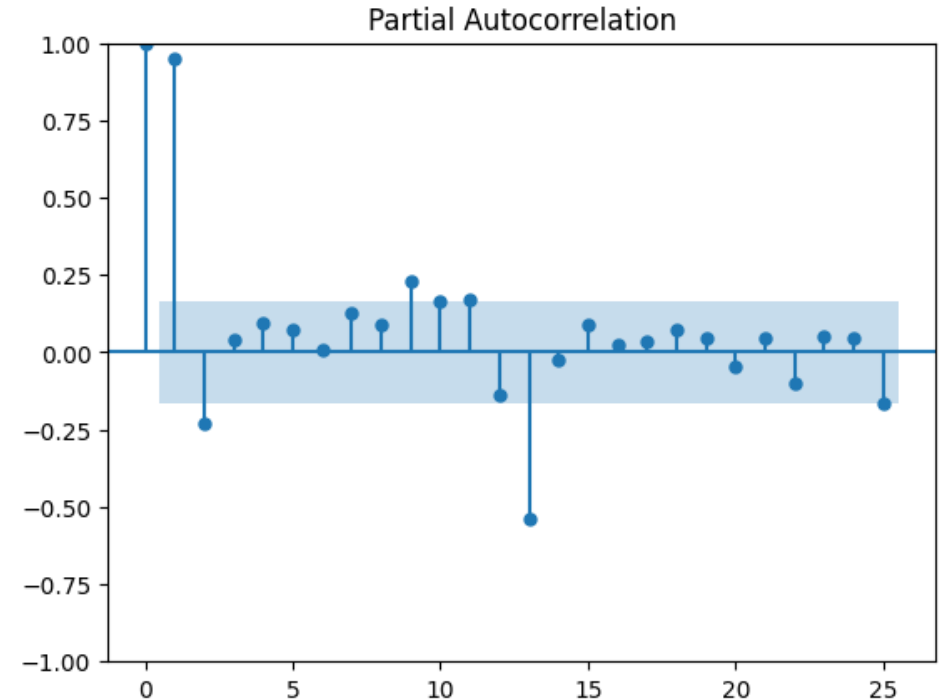
Helps selecting **AR(p)** terms in ARIMA

Typical PACF patterns:

- Shows the *direct* effect of a lag
- Sharp cutoff in PACF → AR process
- Gradual decay in PACF → MA or mixed process

$$x_t = \phi_{k1}x_{t-1} + \phi_{k2}x_{t-2} + \cdots + \phi_{kk}x_{t-k} + \varepsilon_t$$

$$\text{PACF}(k) = \phi_{kk}$$



CODING SESSION

Coming up next in notebook:

- Load energy dataset
- Visualize the data
- Decompose the time-series
- Apply trend and seasonal differentiation



CODING SESSION

CASE STUDY: ELECTRICITY GENERATION

- Measurements: electric power generation using solar, wind and fossil energy sources
- Frequency: monthly sampling rate
- Unit: GW

-> Loading this dataset will provide a multivariate timeseries with 3 columns for each energy source.



QUIZ TIME

Q1. What is a time series?

- ✓ **is a sequence of data points recorded in chronological order.**

Q2. Why is autocorrelation important for time series?

- ✓ **because it measures the extent to which a value in the series is related to previous values.**

Q3. When is a time series stationary?

- ✓ **if its statistical properties, such as mean, variance, and autocorrelation are constant over time**

Q4. Name at least three components into which a time series can be broken down.

- ✓ **Trend, Seasonal component, Residual component**



COFFEE BREAK



RESAMPLING

Resampling in time-series is the process of **changing the frequency** of your time-indexed data.

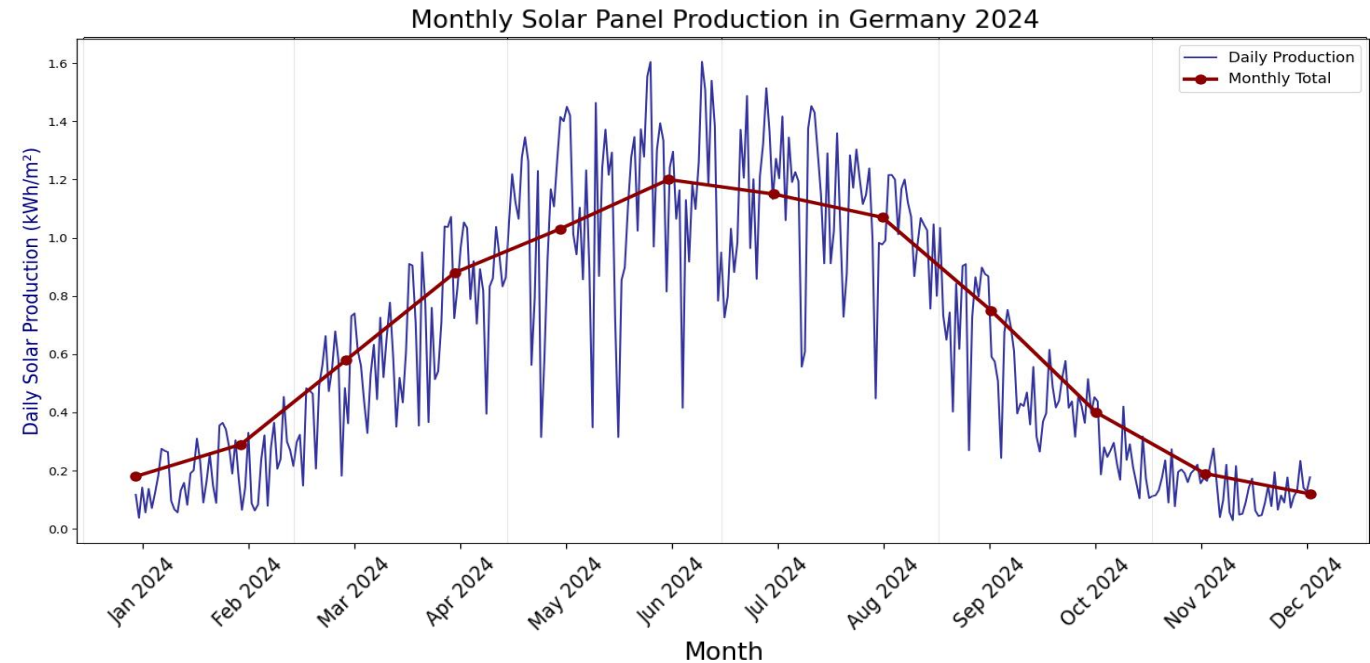
- Downsampling: **aggregating** to a **lower** frequency

$$x_t^{(M)} = \text{Aggregate}(\{x_\tau : \tau \in \text{interval } t\})$$

- Upsampling: **interpolating** to a **higher** frequency

$$x_t^{(H)} = \text{Interpolate}(x_t, x_{t+1})$$

“Resampling changes how frequently we observe the process.”



🤔 **Why?**

- Standardizing time frequency
- Changing the level of detail



HANDLING MISSING VALUES

Missing values in time series are time points where data wasn't recorded, was corrupted, or is otherwise unavailable.

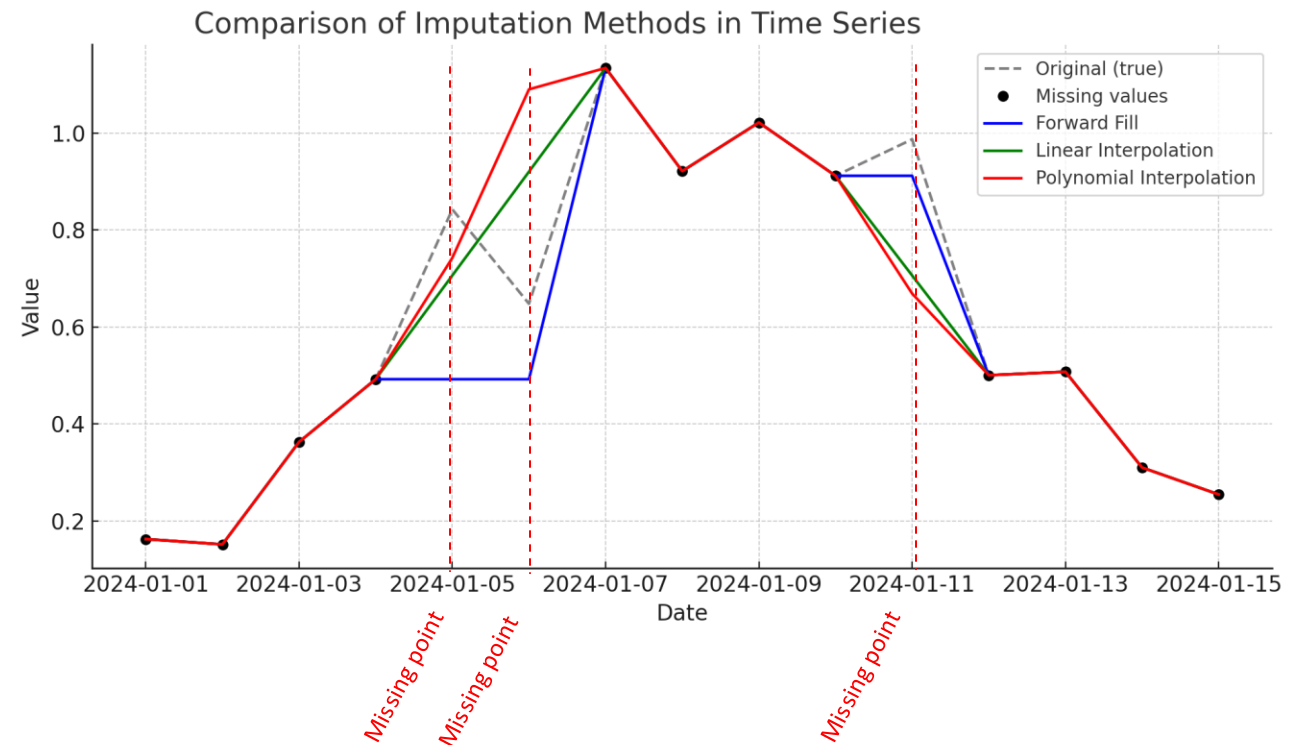
- Forward/Backward Fill:

$$x_t = x_{t-1} \text{ (forward fill)}$$

$$x_t = x_{t+1} \text{ (backward fill)}$$

- Interpolation: estimate missing points based on nearby data using linear/polynomial methods

“Unlike tabular data, we must fill gaps without breaking temporal order.”



Why?

- Handling missing data by respecting **temporal order** and data dynamics



NORMALIZATION

Normalization is the process of transforming the values of the series, so they fit within a **standard scale**, without **distorting** the **temporal** structure.

Common Normalization Methods

- **Min-Max Normalization** (Scaling to $[0, 1]$)

$$x'_t = \frac{x_t - x_{\min}}{x_{\max} - x_{\min}}$$

⚠ Sensitive to outliers

- **Z-Score Normalization** (Standardization)

$$x'_t = \frac{x_t - \mu}{\sigma}$$

where μ = mean, σ = standard deviation.

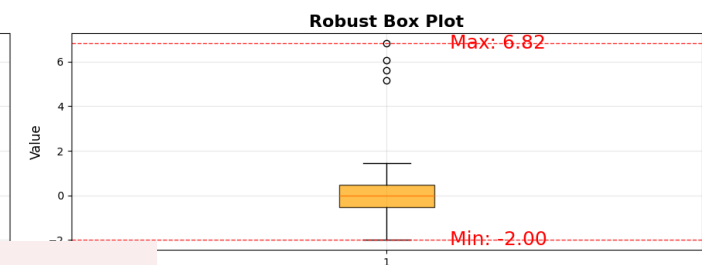
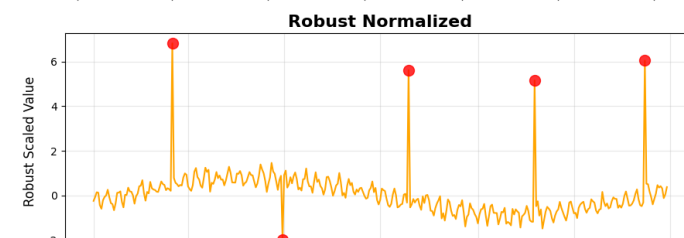
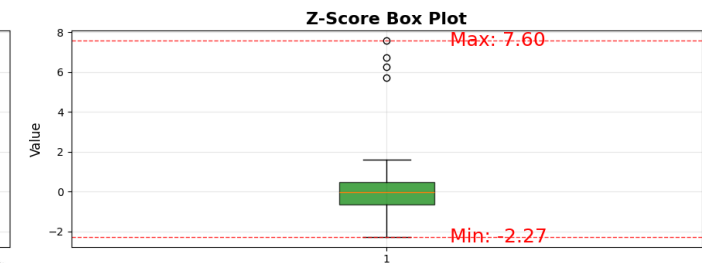
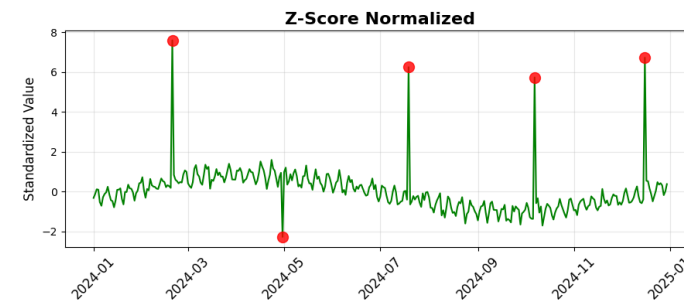
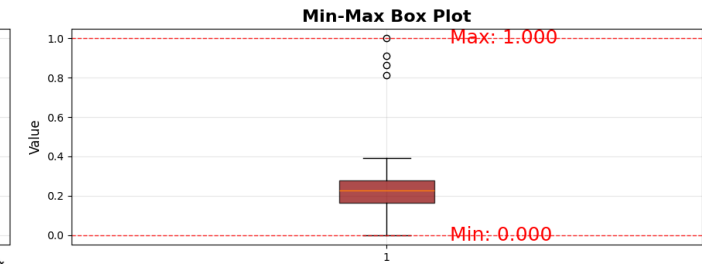
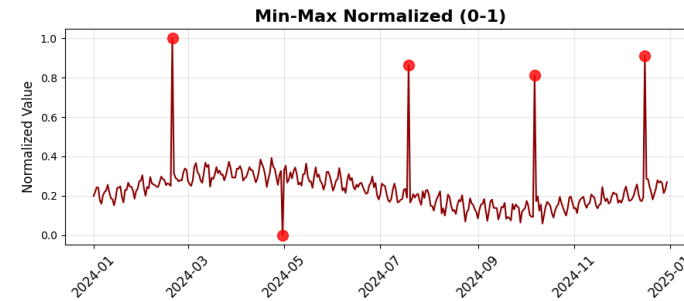
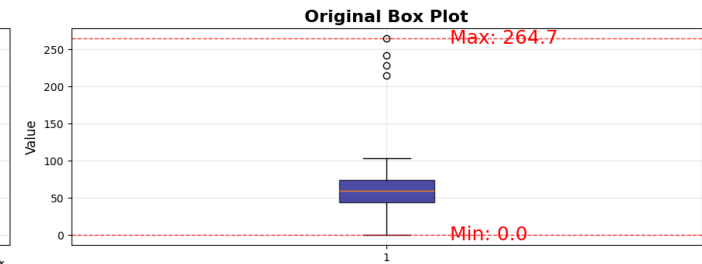
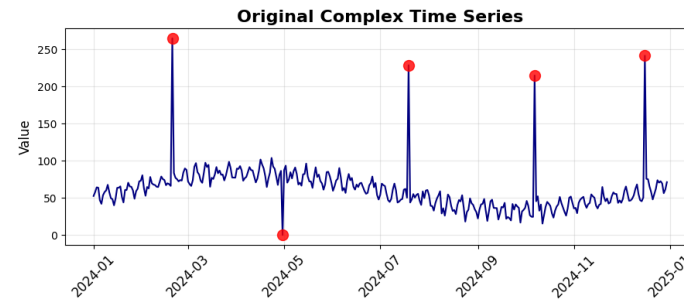
Centers around 0, unit variance

⚠ Common in machine learning for time series

- **Robust Normalization** (Median + IQR)

$$x'_t = \frac{x_t - \text{median}(x)}{\text{IQR}(x)}$$

⚠ Less sensitive to outliers than z-score



🤔 **Why?**

- Faster model convergence, improved robustness
- Makes patterns easier to detect

TRAIN/TEST SPLIT

Key Principles:

- **No random shuffling:** Time series has temporal order. You can't randomly split, or you leak future information into the past.
- **Train on the past, test on the future**

Methods:

- Simple Hold-Out Split: Divide at a single point in time
$$\{x_1, x_2, \dots, x_{T_{train}}\} \text{ (train set)}$$
$$\{x_{T_{train}+1}, \dots, x_T\} \text{ (test set)}$$
- Rolling Window Split: Train window has fixed size, slides forward
- Expanding Window Split: Train set grows over time, test is always the next chunk

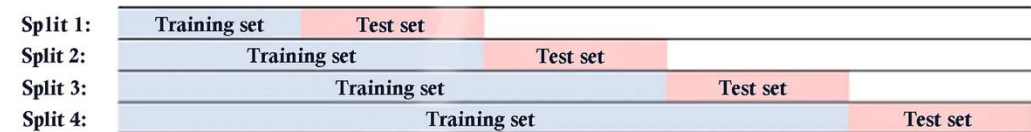
Simple Hold-Out Split



Rolling Window Split



Expanding Window Split



⚠ Typical split: 70–80% training, 20–30% testing

⚠ Common mistake: random split = **data leakage**



CODING SESSION

Coming up next in notebook:

- Check Data Sanity
- Fill missing values
- Apply resampling
- Apply Normalization techniques
- Train/val/test splitting



QUIZ TIME

Q1. Why do we **normalize** time-series data?

- ✓ **to put features on a similar scale without changing the temporal shape.**

Q2. Which normalization method is **most sensitive to outliers**?

- ✓ **Min–Max.**

Q3. Why should we **avoid random splits** in time series?

- ✓ **because it leaks future information into the past**



QUIZ TIME

Q*. Bonus question: What can you see in ACF and PACF plots?

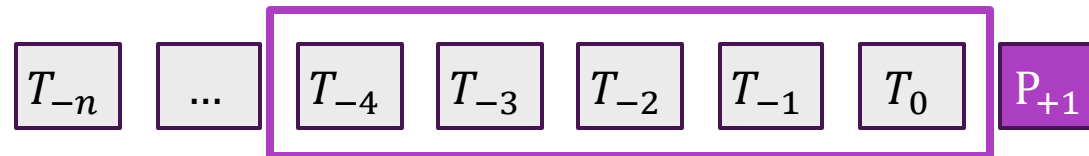
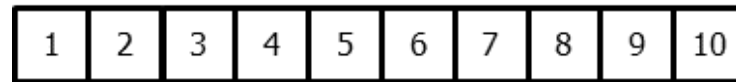
- ✓ **ACF plot:** Shows the correlation between the time series and itself, shifted by different time units (known as lags). The ACF plot provides an overview of the overall correlation between different lags.
- ✓ **PACF plot:** measures the direct relationship between one observation point and another point at a specific lag, after the relationships to the points in between have been adjusted.



TIME-SERIES FORECASTING

Methods:

- One-step Forecaster



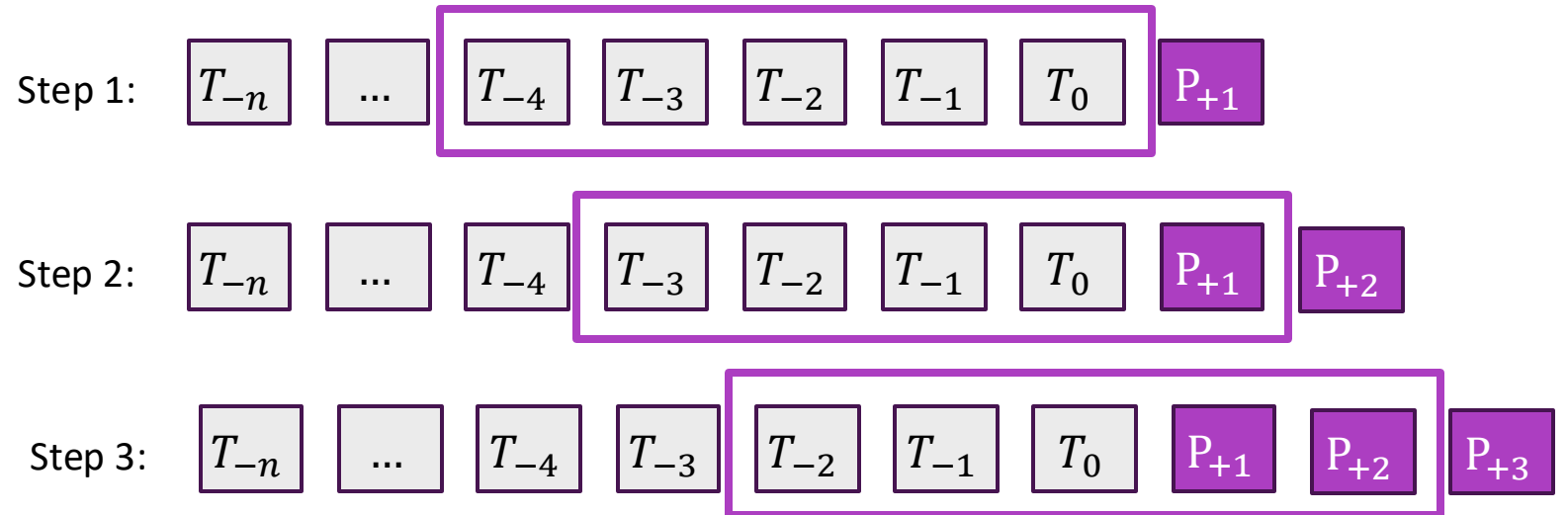
Source to read for more detailed information: <https://www.cienciadedatos.net/documentos/py27-time-series-forecasting-python-scikitlearn.html>



TIME-SERIES FORECASTING

Methods:

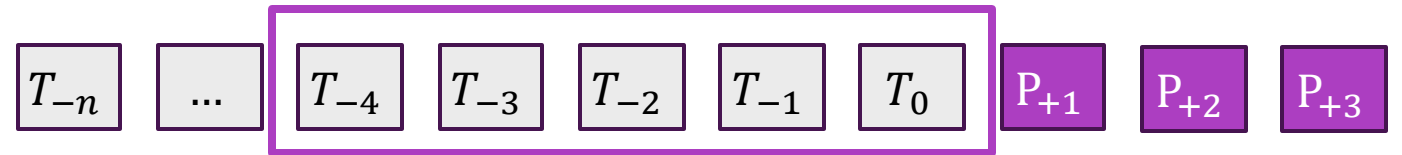
- One-step Forecaster
- **Recursive Multi-Step Forecaster**



TIME-SERIES FORECASTING

Methods:

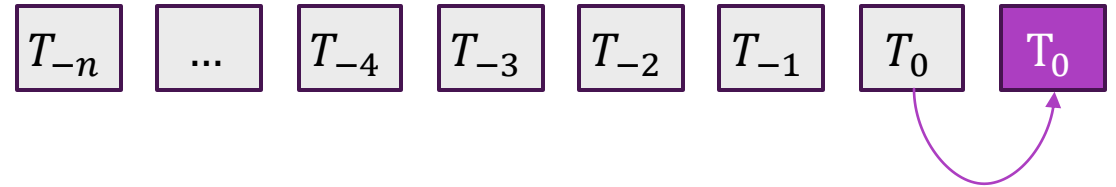
- One-step Forecaster
- Recursive Multi-Step Forecaster
- **One-Shot Multi-Step Forecaster**



BASELINE: NAIVE FORECASTER

A naive forecaster predicts the **next value** by simply repeating the **last observed value**.

$$\hat{y}_{t+1} = y_t$$



Why it matters:

Serves as a **baseline** to judge whether more complex models actually add value



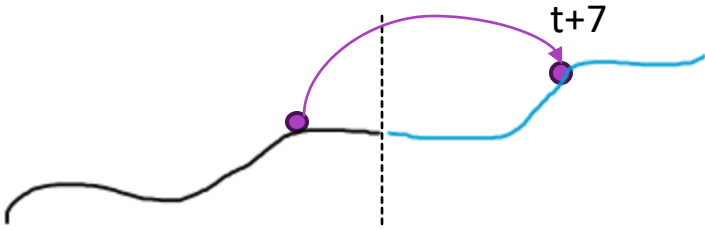
LUNCH BREAK: 12:30-13:30



AUTOREGRESSIVE MODELS

AR <-> PACF

Autoregressive component



Idea: future value depends linearly on a few past values.

$$x_t = \phi_1 x_{t-1} + \phi_2 x_{t-2} + \dots + \epsilon_t$$

ϕ_i : coefficients capturing correlation with *lag i*

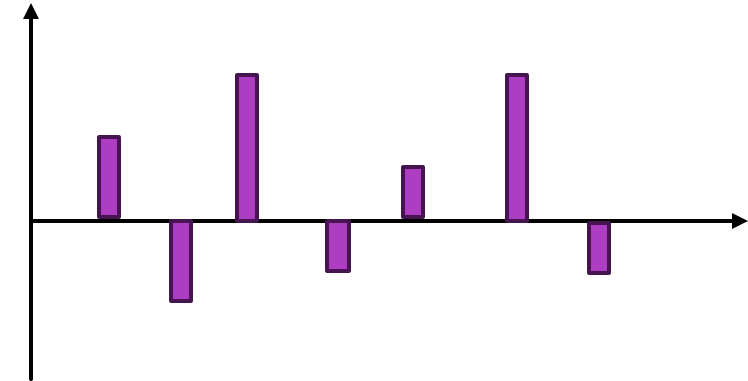
ϵ_t : white noise (random error)

➤ Works well for **stationary series** (constant mean & variance)



MA <-> ACF

Moving Average component



$$x_t = \mu + \phi_1 \epsilon_{t-1} + \phi_2 \epsilon_{t-2} + \dots + \epsilon_t$$

ϕ_i : coefficients capturing correlation with *lag i*

ϵ_t : error terms

μ : mean

EXAMPLE FOR AR AND MA MODELS

	t=-1	t=0	t=1	t=2	t=3	t=4	t=5	t=6	t=7
observed	0	0	-2	-2	-1	2	2	1	0
AR(1) prediction		0							
MA(1) prediction		0							
MA(1) error		0							

$$x_t = 0.5 * x_{t-1}$$

$$x_t = 0.5 (\hat{x}_{t-1} - x_{t-1})$$

dataset with $\mu=0$, $\phi_1 = 0.5$, $\phi_2 = 0.2$



EXAMPLE FOR AR AND MA MODELS

	t=-1	t=0	t=1	t=2	t=3	t=4	t=5	t=6
observed	0	0	-2	-2	-1	2	2	1
AR(2) prediction		0						
MA(2) prediction		0						
MA(2) error	0	0						

$$x_t = 0.5 x_{t-1} + 0.2 x_{t-2}$$

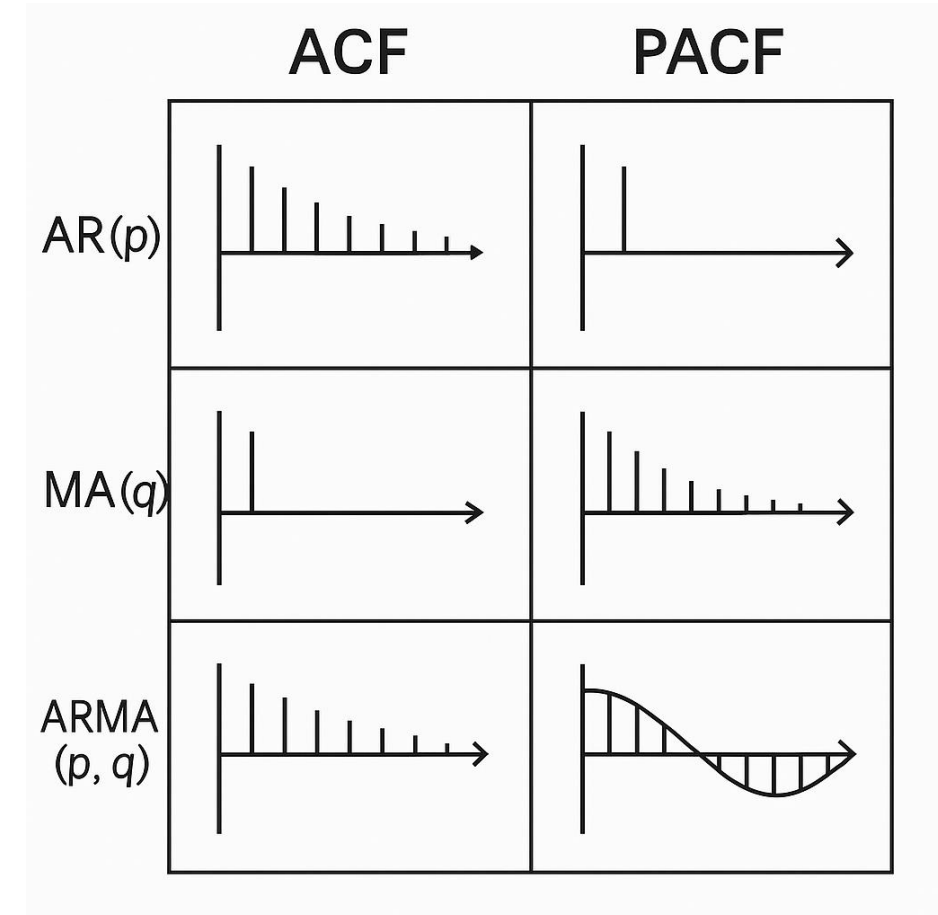
$$x_t = 0.5(\hat{x}_{t-1} - x_{t-1}) + 0.2(\hat{x}_{t-2} - x_{t-2})$$

dataset with $\mu=0$, $\phi_1 = 0.5$, $\phi_2 = 0.2$



ACF & PACF PATTERNS FOR AR, MA, AND ARMA

Model	ACF Pattern	PACF Pattern
AR(p)	Gradual decay Because AR structure propagates correlations through all lags	Sharp cutoff after lag p Only the first p lags have direct influence
MA(q)	Sharp cutoff after lag q Only q past error terms influence the series	Gradual decay Indirect correlations extend further
ARMA(p, q)	Gradual decay influenced by AR & MA structure	Gradual decay No sharp cutoffs due to mixed dependencies



MODEL

AR

(Autoregression)

-> Looking at past values

I

(Integration)

differencing for
handling changes in
data patterns

MA

(Moving Average)

-> considering past errors

Defined by three parameters:

ARIMA(p, d, q)

p : autoregressive terms

d : differencing to remove trend

q : moving-average terms



Widely used for **univariate forecasting** (finance, sales, etc.)



S

(Seasonal)

-> Looking at seasonal patterns

AR

(Autoregression)

-> Looking at past values

I

(Integration)

differencing for handling changes in data patterns

MA

(Moving Average)

-> considering past errors

Defined by three parameters:

autoregressive terms

d: differencing to remove trend

q: moving-average terms

P: seasonal AR terms

D: differencing to remove seasonality

Q: number of lagged forecast errors

m: seasonal period

"SARIMA combines ARIMA with seasonal structure, ideal when the data has strong, regular seasonal patterns"



CODING SESSION

Coming up next in notebook:

- Train ARIMA and SARIMA models



AUTO ARIMA

Advantages:

- Highly **interpretable** and transparent
- Based on **statistical** theory
- Models are **explicitly** defined
- Full manual control over the process

Drawbacks:

- Real-world data rarely remains **stationary**; trends, seasonal patterns, and cycles
- **Computationally demanding** since each time series requires its own model
- **Limited ability** to share insights across multiple time series

